TEMA 9 · ASEGURAMIENTO DE LA CALIDAD DEL SOFTWARE · ITM 2026

Pruebas de una arquitectura basada en contenedores

Integración y ejecución de componentes aislados con Docker Compose



Curso

Aseguramiento de la calidad del software

Integrantes

Juana María Ospina • Jhon Fernando Sánchez • Juliana Velandia • David Santiago Viloria

¿Qué es una arquitectura basada en contenedores?

> **Cada pieza en su propia caja, todas trabajando como un solo sistema.**
>
> Una arquitectura basada en contenedores divide una aplicación en **componentes independientes**, como una API y una base de datos, y cada uno corre **aislado en su propio contenedor** con todo lo que necesita para funcionar.

Conectados

Docker Compose los une con una red interna y les entrega su configuración por variables de entorno.

Con datos persistentes

Los datos se guardan en volúmenes, fuera de los contenedores.

Portables y reemplazables

El sistema se ejecuta igual en cualquier equipo, y cada componente se puede recrear sin afectar a los demás.

¿Por qué probar una arquitectura de contenedores?

Cada componente puede funcionar perfectamente por separado y el sistema completo **fallar de todos modos**:



La API **no encuentra** la base de datos.



La contraseña de la API **no coincide** con la de MySQL.



Los datos **se pierden** al recrear un contenedor.



La base de datos **se cae** y la API se cuelga o devuelve un error que no explica la causa.



No se prueba cada componente aislado, sino las conexiones entre ellos.

A esos lugares de conexión los llamamos **puntos de integración**, y ahí se concentran las pruebas.

Objetivo y las cuatro exigencias del tema

OBJETIVO GENERAL

Evaluar el funcionamiento, la integración, la tolerancia a fallos, la recuperación y la persistencia de una arquitectura compuesta por una API REST y una base de datos MySQL, ejecutadas en contenedores Docker.

Comunicación

¿La API recibe peticiones y habla correctamente con MySQL?

CP-01 · CP-02 · CP-03

Recuperación

Si MySQL cae, ¿la API responde con un error controlado y se recupera sola?

CP-04 · CP-05

Persistencia

¿Los datos sobreviven aunque se destruyan los contenedores?

CP-06

Configuración

¿Los contenedores toman su configuración del entorno y fallan de forma diagnosticable si es incorrecta?

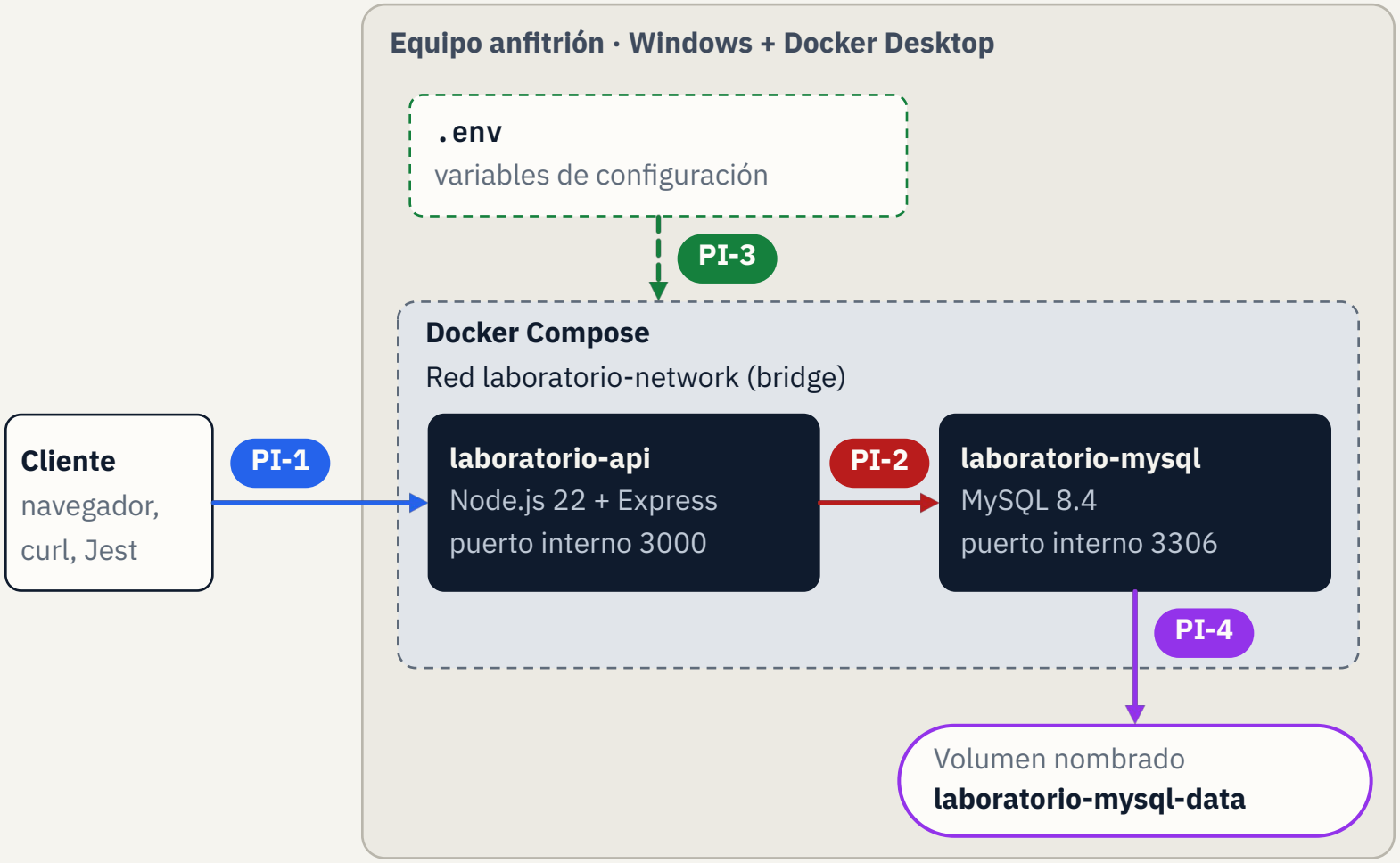
CP-07

Conceptos clave de contenedores

Concepto	Qué es	En el laboratorio
Imagen y contenedor	La imagen es una plantilla de solo lectura; el contenedor es una instancia en ejecución, aislada y efímera	mysql:8.4 (oficial) y la imagen de la API, construida con api/Dockerfile sobre node:22-alpine
Red de Compose	Red privada con DNS interno: cada contenedor encuentra a los demás por el nombre del servicio	Red laboratorio-network (bridge). La API se conecta a mysql:3306, no a localhost ni a una IP
Volumen nombrado	Almacenamiento administrado por Docker fuera del contenedor	laboratorio-mysql-data, montado en /var/lib/mysql. Solo docker compose down -v lo borra
Variables de entorno	Configuración que entra al contenedor desde afuera, sin cambiar código ni imagen	Compose lee el .env y entrega DB_USER, DB_PASSWORD, etc. a cada contenedor
Healthcheck	Comando periódico que distingue «en ejecución» de «listo»	mysqladmin ping cada 2 s; la API solo arranca cuando MySQL está healthy

Dentro del contenedor de la API, localhost es el propio contenedor. Por eso la API usa el nombre de servicio mysql.

Diagrama de arquitectura



CÓMO LEERLO

- **PI-1** HTTP / JSON: el cliente entra desde fuera de Docker por el puerto publicado **3000**.
- **PI-2** SQL: la API llega a MySQL por la red interna usando el **nombre del servicio** mysql.
- **PI-3** El .env llega al **borde de Compose**, que reparte las variables a los dos contenedores.
- **PI-4** El volumen está **fuera** de Compose: sobrevive aunque se destruyan los contenedores.

Componentes del laboratorio

`laboratorio-api`

api/Dockerfile (node:22-alpine)

Recibe peticiones HTTP, valida y consulta MySQL

Anfitrión 3000 → contenedor 3000

Guarda datos: no (efímero)

`laboratorio-mysql`

mysql:8.4 (imagen oficial)

Guarda la tabla productos

Anfitrión 3313 → contenedor 3306

Guarda datos: no por sí mismo, usa el volumen

`laboratorio-network`

Red bridge creada por Compose

Conecta los contenedores y resuelve sus nombres

Guarda datos: no

`laboratorio-mysql-data`

Volumen nombrado

Conserva los archivos de la base de datos

Guarda datos: sí

`.env`

Copia de .env.example (no se sube al repositorio)

Puertos, usuario, contraseñas y nombre de la base

`mysql/init.sql`

Script del repositorio

Crea la tabla e inserta 3 productos semilla, **solo si el volumen está vacío**

Pila tecnológica: Node.js 22 · Express 5 · mysql2 (pool de 10 conexiones) · MySQL 8.4 · Docker y Docker Compose · Jest 30 · PowerShell · Git

Los cuatro puntos de integración

ID	Integración	Qué viaja	Qué lo define en docker-compose.yml
PI-1	**Anfitrión → API**	Peticiones HTTP y respuestas JSON con su código de estado	`ports: "${API_PORT}:3000"`

PI-2	**API → MySQL**	Consultas SQL (SELECT, INSERT) y sus resultados	`red laboratorio-network`
			`DB_HOST: mysql · DB_PORT: 3306`
			`depends_on: service_healthy`

PI-3	**.env → contenedores**	Host, puerto, base de datos, usuario y contraseña	`environment de cada servicio con ${...}`
PI-4	**MySQL → volumen**	Los archivos de datos de MySQL	`volumes: mysql-data:/var/lib/mysql`

PI-2 (API → MySQL) es el punto más crítico: lo cubren 5 de los 7 casos, porque es donde la API depende de otro contenedor que puede no estar disponible.

La API y su contrato de respuestas

Método y ruta	Respuestas posibles
GET /health	200 {"estado":"OK","baseDatos":"conectada"} · 503 si MySQL no responde (ejecuta SELECT 1)
GET /productos	200 con la lista · 503 si MySQL no responde
GET /productos/:id	200 · 400 si el id no es un entero positivo · 404 si no existe · 503 si MySQL no responde
POST /productos	201 con el producto creado · 400 si los datos son inválidos · 503 si MySQL no responde

Validación de POST /productos: el nombre es un texto no vacío de hasta 100 caracteres; el precio es un número finito mayor que 0 y hasta 99 999 999.99, redondeado a dos decimales. Las consultas son parametrizadas (?).

CADA CÓDIGO SIGNIFICA ALGO DISTINTO

400 — **El cliente envió algo inválido**
Nombre vacío, precio negativo, JSON mal formado

404 — **MySQL respondió, pero el recurso no existe**
GET /productos/999999999

503 — **Una dependencia no está disponible**
MySQL detenido o credenciales incorrectas

500 — **Error inesperado**
Cualquier otro error no controlado

> **Decisión de diseño: cuando MySQL falla, la API no se cae. Sigue viva y responde 503, que distingue «la base falló» de «el producto no existe» (404).**

Decisiones de arquitectura

Decisión	Motivo
Dos contenedores: API y base de datos	Es el mínimo que permite probar una integración real
Versiones fijas: node:22-alpine y mysql:8.4	Todos los integrantes ejecutan exactamente lo mismo
container_name fijo	Permite inspeccionar y detener contenedores por nombre en las pruebas
Puerto externo 3313 para MySQL	Evita conflictos con un MySQL instalado localmente en el 3306
Usuario de aplicación laboratorio_user	La API no usa root; root queda para administración
Volumen nombrado	Los datos no dependen del ciclo de vida del contenedor
Healthcheck + depends_on: service_healthy	La API solo arranca cuando MySQL acepta conexiones
Configuración en .env	La misma imagen sirve con otra configuración sin cambiar el código
Respuesta 503 ante fallas de MySQL	Error controlado y diagnosticable en lugar de una caída

Cierre del bloque 1: dos componentes aislados, Docker Compose como «pegamento» (red, volumen y configuración) y cuatro puntos de integración que vamos a probar.

Enfoque de la estrategia

NIVEL DE PRUEBA

Integración

Valida que dos componentes se comuniquen e intercambien información correctamente.

NIVEL DE PRUEBA

Sistema

Valida el comportamiento completo en un flujo de principio a fin.

Ninguna dependencia es simulada. API, MySQL, red y volumen son reales. Simular MySQL dejaría sin probar justamente la red, la configuración y el volumen.

Las fallas se provocan con comandos reales de Docker: docker compose stop, down, up y up --force-recreate.

LAS PRUEBAS SE LANZAN DESDE EL ANFITRIÓN CONTRA HTTP://LOCALHOST:3000

Anfitrión → **Contenedor de la API** → **Red** → **Contenedor de MySQL**

Trazabilidad: de la exigencia a la evidencia

Exigencia → Punto de integración → Riesgo → Caso de prueba → Evidencia

Exigencia	PI	Riesgo	Casos	Evidencia
Comunicación	PI-1, PI-2	RG-1: la API no es alcanzable o responde con un contrato distinto (otro código u otro JSON)	CP-01, CP-02, CP-03	CP-01-03-funcional.txt
Recuperación	PI-2	RG-2: MySQL cae o no está listo, y la API se cuelga o diagnostica mal	CP-04, CP-05 (y CP-01 a CP-03)	CP-04-05-falla.txt
Persistencia	PI-4	RG-4: los datos se pierden al recrear el contenedor	CP-06	CP-06-persistencia.txt
Configuración	PI-3	RG-3: una variable errónea deja a la API sin acceso a la base	CP-07	CP-07-configuracion.txt

Cada riesgo tiene al menos un caso y cada caso tiene su archivo de evidencia.

Alcance y exclusiones

Incluye

- Pruebas de integración entre la API y MySQL
- Pruebas de sistema: caída y recuperación, persistencia y configuración
- Ejecución local con Docker Compose

Se excluye	Motivo
Carga y rendimiento	Corresponden a otro tema de la guía (tema 15)
Seguridad	Credenciales de prueba en un laboratorio local
Pruebas unitarias	El tema trata la integración, no funciones aisladas
Interfaz gráfica	El laboratorio no tiene interfaz; todo se prueba por HTTP
Orquestación en varios nodos	Un solo equipo, sin Kubernetes ni réplicas

Ambiente, herramientas y datos de prueba

AMBIENTE

Windows con Docker Desktop, Node.js 22, PowerShell, archivo .env y puertos 3000 y 3313 libres.

PRECONDICIÓN DE TODA LA SUITE

```
docker compose up -d --build
```

Los dos contenedores en ejecución y MySQL en estado healthy.

Herramienta	Uso	Casos
Docker Compose	Levantar el ambiente y provocar las fallas	Todos
Jest + fetch	Pruebas automatizadas con aserciones	CP-01 a CP-05
PowerShell	Scripts que recrean el entorno y cambian la configuración	CP-06, CP-07

DATOS DE PRUEBA

- **Todos son ficticios.** Semilla: Teclado, Mouse y Monitor (init.sql).
- **Cada caso crea su propio producto con un nombre único** (marca de tiempo o GUID) y lo verifica **por su id**, nunca por la cantidad total. Así la suite se repite sin reiniciar la base de datos.

Id inexistente: 999999999

Contraseña errónea: clave_incorrecta_CP07, definida solo en la terminal

Matriz resumen de los 7 casos

Totales: 7 casos · 4 de integración y 3 de sistema · 5 automatizados y 2 con script.

ID	Nombre	Tipo	PI	Riesgo	Ejecución	Comando o script
CP-01	Crear y consultar un producto	Integración	PI-1, PI-2	RG-1, RG-2	Jest	npm run test:funcional
CP-02	Rechazar datos inválidos	Integración	PI-1, PI-2	RG-1, RG-2	Jest	npm run test:funcional
CP-03	Consultar un producto inexistente	Integración	PI-1, PI-2	RG-1, RG-2	Jest	npm run test:funcional
CP-04	Caída de MySQL	Integración	PI-2	RG-2	Jest	npm run test:falla
CP-05	Recuperación tras la caída	Sistema	PI-2	RG-2	Jest	npm run test:falla
CP-06	Persistencia al recrear contenedores	Sistema	PI-4	RG-4	Script	cp06-persistencia.ps1
CP-07	Configuración por entorno	Sistema	PI-3	RG-3	Script	cp07-configuracion.ps1

Mínimos de la guía cubiertos

Un escenario exitoso
CP-01

Un escenario de validación o error funcional
CP-02, CP-03

Un escenario de falla técnica o de integración
CP-04, CP-05

Casos de comunicación: CP-01, CP-02 y CP-03

Caso	Qué se hace	Qué se espera	Detalle importante
CP-01 Crear y consultar un producto	POST /productos con precio 12500.567 y luego GET /productos/<id>	**201** con id; **200** con el mismo nombre y precio 12500.57	Comprueba el **redondeo a dos decimales** y que MySQL guarda y devuelve lo mismo
CP-02 Rechazar datos inválidos	Enviar **8 entradas inválidas** y contar los productos antes y después	**400** en todas; la cantidad de productos no cambia	**Ningún dato inválido llega a la base: la validación ocurre antes del INSERT**
CP-03 Consultar un producto inexistente	GET /productos/999999999	**404** con «Producto no encontrado»	**Es 404 y no 503, porque MySQL sí respondió (sin filas)**

LAS 8 ENTRADAS INVÁLIDAS DE CP-02

Cuerpo vacío	Nombre vacío	Sin precio	Precio 0
Precio negativo	Precio como texto ("100")	Precio mayor que 99999999.99	Nombre de 107 caracteres

Casos de recuperación: CP-04 y CP-05

1 · Antes de la caída

`GET /health`

La API ejecuta SELECT 1 en MySQL

Responde 200

→

2 · MySQL detenido · CP-04

`docker compose stop mysql`

- GET /health y GET /productos → **503**
- Conexión rechazada: la API no llega a MySQL
- docker inspect: la API **sigue en ejecución** (State.Running = true)

→

3 · MySQL de vuelta · CP-05

`docker compose up -d mysql`

- MySQL se inicia con el mismo volumen
- GET /health cada 2 s hasta obtener **200**
- El pool de conexiones se reconecta solo
- Mismo id del contenedor de la API antes y después
- GET /productos/:id → **200** con el producto creado antes de la caída

La API se recupera sola. No hubo que reiniciarla: el pool de mysql2 vuelve a conectarse cuando MySQL regresa.

Además, la prueba siempre vuelve a iniciar MySQL al final (afterAll), aunque falle, para no dejar el sistema caído.

Persistencia y configuración: CP-06 y CP-07

CP-06 Persistencia al recrear contenedores

1. Crea un producto (CP-06-Persistencia-<GUID>, precio 35000) y guarda su id.
2. docker compose down → **destruye** los contenedores y la red, pero **no** el volumen.
3. docker compose up -d → crea contenedores nuevos que montan el mismo volumen.
4. GET /productos/<id> → el producto sigue ahí con el mismo nombre y precio.

CP-05 no reemplaza a CP-06: stop conserva el contenedor; solo down lo destruye. CP-06 es el que demuestra que el dato vive en el volumen.

CP-07 Configuración por entorno

1. Define DB_PASSWORD=clave_incorrecta_CP07 **solo en la terminal**. Una variable de la terminal tiene **prioridad** sobre el .env.
2. Recrea únicamente la API: docker compose up -d --force-recreate api.
3. Verifica **503** en /health y /productos, y **Access denied** en el log de la API.
4. Quita la variable, recrea la API y verifica **200**.

La misma imagen, el mismo código y el mismo .env: solo cambió una variable de entorno, y el sistema pasó de funcionar a fallar de forma controlada y diagnosticable.
El script restaura la variable al terminar, incluso si falla.

Criterios de aprobación

Un caso se aprueba si se cumplen las tres condiciones:

1
El **código HTTP** coincide con el esperado.

2
El **cuerpo de la respuesta** coincide (campos y mensaje de error).

3
Donde aplica, el **estado de la base de datos** también coincide (un dato inválido no se guardó o el producto sigue existiendo).

El tiempo agotado es una falla: una respuesta que no llega no se aprueba.

CRITERIO DE ENTRADA
Ambiente levantado y MySQL en estado healthy.

CRITERIO DE SALIDA
Los 7 casos ejecutados, cada uno con su evidencia en evidencias/ y su estado en la matriz.

Qué vamos a demostrar y en qué orden

1. **Entorno levantado** — docker compose ps
2. **Comunicación** — CP-01 a CP-03
3. **Caída y recuperación** — CP-04 y CP-05
4. **Persistencia** — CP-06
5. **Configuración** — **CP-07 · siempre al final**
6. **Evidencias** — carpeta evidencias/

CP-07 va al final porque es el único caso que deja a la API sin conexión a propósito.

CP-06 va antes porque recrea todo el entorno.

PLAN B

Si Docker o la red fallan: archivos de evidencias/, matriz de casos o video de respaldo.

Guion de la demostración paso a paso

#	Paso	Comando	Resultado esperado	Tiempo
1	**Mostrar el entorno levantado**	`docker compose ps`	laboratorio-mysql en Up (healthy) y laboratorio-api en Up	≈ 1 min
2	**API viva y conectada**	`curl.exe -i http://localhost:3000/health`	HTTP/1.1 200 OK · baseDatos: conectada	≈ 30 s
3	**Comunicación · CP-01 a CP-03**	`cd api` `npm run test:funcional`	Tests: 3 passed, 3 total (≈ 20 s)	≈ 1 min
4	**Caída y recuperación · CP-04 y CP-05**	`npm run test:falla`	MySQL se detiene e inicia dos veces · Tests: 2 passed, 2 total (≈ 30 s)	≈ 1,5 min
5	**Persistencia · CP-06 (desde la raíz)**	`powershell -ExecutionPolicy Bypass -File .\scripts\cp06-persistencia.ps1`	Producto creado: ID=<n> · RESULTADO: APROBADO (≈ 12 s)	≈ 1 min
6	**Configuración · CP-07 (siempre al final)**	`powershell -ExecutionPolicy Bypass -File .\scripts\cp07-configuracion.ps1`	HTTP 503 y Access denied en el log · RESULTADO: APROBADO (≈ 10 s)	≈ 1 min
7	**Mostrar las evidencias**	`Get-ChildItem .\evidencias -File | Select-Object Name, Length, LastWriteTime`	Archivos de evidencias/ con nombre, tamaño y fecha	≈ 30 s

Advertencias y plan B

⚠ Nunca ejecutar docker compose down -v

Borra el volumen y todos los datos; además, init.sql volvería a crear solo los 3 productos semilla.

Texto rojo NativeCommandError al correr npm en PowerShell

Qué hacer: es ruido; npm escribe en la salida de error. Lo que vale es la línea Tests: … passed.

Invoke-RestMethod muestra un error con 503

Qué hacer: es el comportamiento esperado de PowerShell ante códigos de error; para mostrar el código, usar curl.exe -i.

La API no responde justo después de levantar el entorno

Qué hacer: esperar a que MySQL esté healthy (docker compose ps).

Docker o la red fallan durante la exposición

Plan B: mostrar los archivos de evidencias/ y la matriz de casos, o el video de respaldo.

Resultados: 7 de 7 casos aprobados

7/7 casos aprobados

4 exigencias cubiertas

4 puntos de integración

4 riesgos cubiertos

5 + 2 automatizados + con script

Caso	Resultado observado	Tiempo	Estado
CP-01	POST → 201 con id; GET → 200 con el mismo nombre y precio 12500.57	112 ms	Aprobado
CP-02	Las 8 entradas inválidas → 400; la cantidad de productos no cambió	64 ms	Aprobado
CP-03	404 con «Producto no encontrado»	9 ms	Aprobado
CP-04	/health y /productos → 503; la API siguió en ejecución	14,2 s	Aprobado
CP-05	/health volvió a 200; mismo contenedor de la API; el producto previo respondió 200	14,8 s	Aprobado
CP-06	El producto ID=20 siguió consultable con el mismo nombre y precio (35000)	12 s	Aprobado
CP-07	Contraseña errónea: 503 y Access denied en el log. Restaurada: 200	10 s	Aprobado

Hallazgos: defectos de integración corregidos

Al ejecutar el laboratorio **antes de los ajustes**, aparecieron dos defectos reales. Ninguno estaba «en una función»: ambos estaban en **cómo se conectan los componentes**.

Hallazgo	Antes (observado)	Corrección	Después
H1 Riesgo RG-2	Justo después de docker compose up, la API registraba ECONNREFUSED ...:3306 y respondía 503 durante unos segundos: depends_on esperaba a que MySQL **arrancara**, no a que **aceptara conexiones**.	Healthcheck de MySQL (mysqladmin ping cada 2 s) y depends_on: condition: service_healthy	La API arranca solo cuando MySQL está listo
H2 Riesgo RG-2	Con MySQL detenido, /health seguía respondiendo 200 «OK»: **reportaba un estado falso**.	/health ejecuta SELECT 1 y responde 503 si MySQL no contesta	/health dice la verdad sobre la dependencia

Lección: «en ejecución» no es lo mismo que «listo», y un endpoint de salud que no revisa sus dependencias da una falsa sensación de seguridad.

Hallazgos: comportamientos confirmados

Hallazgo	Qué confirmaron las pruebas	Evidencia
H3	**La API se recupera sola** cuando MySQL vuelve: el pool de conexiones se reconecta sin reiniciar el contenedor	CP-05: mismo id de contenedor antes y después
H4	**Los errores distinguen la causa:** 400 (dato inválido), 404 (no existe), 503 (la dependencia falló)	CP-02, CP-03, CP-04, CP-07
H5	**Ningún dato inválido llega a MySQL:** la validación ocurre antes del INSERT	CP-02: conteo igual antes y después
H6	**Los datos viven en el volumen, no en el contenedor**	CP-06: el producto ID=20 sobrevivió a down y up
H7	**La comunicación por nombre de servicio funciona:** Access denied for user 'laboratorio_user'@'172.18.0.3' solo lo puede emitir MySQL, así que la API lo alcanzó por la red y falló después, en la autenticación	CP-07: log de la API
H8	**La configuración externa decide el comportamiento:** con la misma imagen, una sola variable hizo fallar el sistema de forma diagnosticable	CP-07

Limitaciones

Limitación	Qué no se puede concluir
Sin pruebas de carga ni rendimiento (tema 15)	Cuántas peticiones soporta la API ni cómo se degrada
Sin pruebas de seguridad	Que la configuración sea segura: contraseñas en texto plano en .env, root accesible por el 3313, sin TLS
Un solo equipo, sin orquestador	Alta disponibilidad, balanceo o tolerancia a fallos entre nodos
Una sola corrida documentada	Que la secuencia completa sea repetible en otros equipos
Tiempos de CP-04 y CP-05 incluyen la espera de Docker	El tiempo real de detección y de recuperación de la API
CP-06 y CP-07 son scripts de PowerShell	No están en npm test y dependen de Windows
Solo se probó la caída de MySQL	Qué pasa si se cae el contenedor de la API
CP-07 solo prueba la contraseña	Otros errores de configuración (DB_HOST, DB_NAME, puerto)

Nota: el repositorio incluye tres scripts complementarios de carga en api/test/ (carga.js, degradacion.js y recuperacion-carga.js). No forman parte de los 7 casos y no tienen evidencia guardada, por lo que no se usan en los resultados.

Mejoras propuestas

1. **Medir por separado** el tiempo de detección de la caída y el de recuperación (marca de tiempo en cada sondeo).
2. **Migrar CP-06 y CP-07** a pruebas automatizadas multiplataforma e incluirlas en npm test.
3. **Afirmar también el cuerpo del error** en CP-02 y CP-04, no solo el código HTTP.
4. **Comprobar explícitamente** en CP-06 que init.sql no se volvió a ejecutar.
5. **Reportes estructurados** (por ejemplo, jest --json o JUnit) y evidencias en UTF-8.
6. **Integración continua:** ejecutar la suite con Docker Compose en un pipeline y repetirla varias veces.
7. **Ampliar la tolerancia a fallos:** política restart para la API y un caso de caída del contenedor de la API.
8. **Endurecer la seguridad:** manejo de secretos, no exponer root y ejecutar la imagen con un usuario sin privilegios.

Conclusiones

1. **Los fallos están en las conexiones entre componentes.**
2. **Hay que probar qué pasa cuando algo falla, no solo cuando todo funciona.**
3. **La configuración externa también se tiene que probar.**

CIERRE

> La calidad de una arquitectura basada en contenedores se demuestra probando sus conexiones: comunicación, configuración, persistencia y recuperación

¿Preguntas?

Preguntas probables y respuestas sugeridas

Pregunta	Respuesta sugerida
¿Por qué no simularon MySQL con un mock?	Porque el objetivo es la integración. Un mock dejaría sin probar la red, la configuración y el volumen, que es justamente donde aparecen las fallas.
¿Por qué 503 y no 500?	503 indica que una dependencia no está disponible temporalmente; 500 es un error interno inesperado. Distinguirlos permite diagnosticar y reintentar.
¿Cómo saben que la API no se reinició en CP-05?	Se compara el id del contenedor (docker inspect) antes y después de la caída: es el mismo. En CP-04 también se verifica State.Running = true.
¿Por qué CP-05 no demuestra la persistencia?	Porque stop conserva el contenedor. Solo down lo destruye, y eso es lo que hace CP-06.
¿Por qué la API usa mysql y no localhost?	Dentro del contenedor, localhost es el propio contenedor. El DNS interno de Docker resuelve el nombre del servicio mysql.
¿Para qué el healthcheck si ya existe depends_on?	depends_on solo espera a que el contenedor arranque. Con condition: service_healthy espera a que MySQL acepte conexiones (hallazgo H1).

ANEXO C

Referencias principales

Docker Inc. (s. f.). *Compose file reference*. https://docs.docker.com/reference/compose-file/

Docker Inc. (s. f.). *Control startup and shutdown order in Compose*. https://docs.docker.com/compose/how-tos/startup-order/

Docker Inc. (s. f.). *Networking overview*. https://docs.docker.com/engine/network/

Docker Inc. (s. f.). *Volumes*. https://docs.docker.com/engine/storage/volumes/

Wiggins, A. (2017). *The Twelve-Factor App: III. Config*. https://12factor.net/config

Souppaya, M., Morello, J. y Scarfone, K. (2017). *Application container security guide* (NIST SP 800-190). https://doi.org/10.6028/NIST.SP.800-190

International Software Testing Qualifications Board. (s. f.). *ISTQB Glossary*. https://glossary.istqb.org/

Sommerville, I. (2016). *Software engineering* (10.ª ed.). Pearson.

La lista completa, en formato APA 7, está en docs/referencias.md.